

Linux 文件 & 文本Files & Text	
导航 / 文件	
ls -lah	详情 + 隐藏 + 人类可读
cd -	回到上一目录
cp -r src/ dst/	递归复制目录
mv a b	.
rm -rf	移动 / 强制递归删
ln -s target link	软链接
find . -name "*.log" -mtime -7	7 天内日志
find . -size +100M -delete	删大文件
查看 / 编辑	
cat	.
less	.
head -n 20	看 / 翻页 / 取头
tail -f log	实时跟踪日志
wc -l file	行数
搜索 / 替换	
grep -rn "key" src/	递归带行号搜
grep -E "a b"	.
-i	.
-v	扩展正则 / 忽大小写 / 反选
sed -i 's/old/new/g' f	原地替换(macOS 加 '')
awk '{print \$1, \$3}'	取列
cut -d, -f1,3	.
sort -u	.
uniq -c	切列 / 去重 / 计数

Git 基础Git Basics	
配置 / 初始化	
git config --global user.name "X"	设全局名
git init	.
clone url	初始化 / 克隆
日常	
git status	看变更
git add -p	交互式分块暂存
git commit -m "msg"	提交
git commit --amend	改最近一次提交
git diff	.
--staged	工作区 / 暂存区差异
git log --oneline --graph --all	看图
git show HEAD	看具体 commit
分支	
git switch -c feat/x	建并切(新)
git switch main	切分支
git merge feat/x	合入当前分支
git rebase main	变基,线性历史
git rebase -i HEAD~5	交互式 squash/reorder
远程	
git fetch	.
pull --rebase	拉 / 拉并变基
git push -u origin feat/x	首推 + 设上游

进程 & 系统Process & System	
权限	
chmod 755 f	.
+x	rxwx r-x r-x / 加执行
chown user:group f	改属主
sudo !!	用 sudo 重跑上条
进程 / 资源	
ps aux grep nginx	查进程
top	.
htop	.
btop	交互式监控
kill -9 PID	强杀(SIGKILL)
killall -f "pattern"	按命令行匹配杀
nohup cmd &	后台不挂断
jobs	.
fg %1	.
bg	作业控制
磁盘 / 网络	
df -h	.
du -sh *	分区 / 当前目录占用
free -h	内存
lsof -i :8080	占端口的进程
ss -tlnp	监听端口(替代 netstat)
curl -fsSL url	静默 + 跟随 + 失败退出
scp f user@host:/path	拷贝到远端
rsync -avz src/ host:dst/	增量同步

Git 撤销 & 进阶Undo & Advanced	
撤销变更	
git restore file	丢工作区改动
git restore --staged file	从暂存区撤回
git reset --soft HEAD ~1	撤 commit 留改动
git reset --hard HEAD ~1	彻底丢弃(危险)
git revert <sha>	反向提交,保留历史
救命	
git reflog	所有 HEAD 移动,丢 commit 找回
git stash	.
pop	临时藏匿改动
git cherry-pick <sha>	挑某个 commit 到当前分支
侦探	
git blame file	每行谁改的
git log -S "text"	谁加/删了这段字符串
git bisect start	二分定位坏 commit
协作礼仪	
提交信息: type(scope): subject(feat/fix/docs/refactor)	
已 push 的不要 force--push,除非用 --force-with-lease	
主分支保护 + PR + CI,禁止直推	
.gitignore 写早,误提秘密用 git filter-repo	

管道 & 变量Pipes & Vars	
重定向 / 管道	
cmd > out 2>&1	合并 stderr 到 stdout
cmd >> log	追加
cmd1 cmd2	前者 stdout → 后者 stdin
cmd tee out	分流到文件 + 屏幕
find . xargs rm	把行作参数批量执行
<(cmd)	.
\$(cmd)	进程替换 / 命令替换
变量 & 引号	
"\$var"	vs
'\$var'	双引号展开,单引号字面
\${var:-default}	空时取默认
\${#var}	长度
\${var/a/b}	长度 / 替换
export VAR=val	导出到子进程
脚本严格模式	
#!/usr/bin/env bash set -euo pipefail IFS=\$'\n\t'	
实用单行	
!!	.
!\$	.
!*	上条 / 上条未参 / 上条全参
Ctrl-R	反向搜历史
watch -n 1 'cmd'	每秒重跑

Docker 镜像 & 容器Images & Containers	
镜像	
docker pull img:tag	拉镜像
docker build -t name:tag .	构建
docker images	.
rmi id	列 / 删
docker tag a b:v1	打标
docker push registry/img	推到仓库
容器	
docker run --rm -it img sh	用完即删,交互
-d	.
--name x	.
-p 8080:80	后台 / 命名 / 端口映射
-v \$PWD:/app	.
-e KEY=val	挂卷 / 环境变量
--restart unless-stopped	自动重启
docker exec -it ct bash	进运行中容器
docker logs -f --tail 100 ct	看日志
docker ps -a	列全部含已停
docker stats	实时资源
Dockerfile 关键	
FROM	.
WORKDIR	.
COPY	基础 / 工作目录 / 拷贝
RUN	.
CMD	.
ENTRYPOINT	构建命令 / 默认参数 / 入口
HEALTHCHECK	.

SSH & 远程SSH	
连接	
ssh user@host	基础连接
ssh -p 2222 user@host	指定端口
ssh -i ~/.ssh/id_xx user@h	指定私钥
密钥	
ssh-keygen -t ed25519	现代算法,首选
ssh-copy-id user@host	推公钥到远端
ssh-add ~/.ssh/key	加到 agent
~/.ssh/config 模板	
Host prod	
HostName 1.2.3.4	
User deploy	
Port 22	
IdentityFile ~/.ssh/prod_ed25519	
ServerAliveInterval 60	
隧道 / 转发	
ssh -L 8080:localhost:80 h	本地 → 远端服务
ssh -R 9000:localhost:3000 h	远端 → 本地(反向)
ssh -D 1080 host	SOCKS 代理
ssh -J jump host	跳板机
实用	
ssh host 'cmd'	远程执行不进 shell
tmux / screen	断线不丢会话

Compose & 运维Compose & Ops	
compose 命令	
docker compose up -d	起所有服务,后台
compose down -v	停 + 删 + 删卷
compose logs -f svc	跟某服务日志
compose exec svc sh	进服务容器
compose restart svc	只重启一个
compose.yaml 模板	
services:	
api:	
build:	
ports: ["8080:80"]	
environment:	
DATABASE_URL: \${DB_URL}	
depends_on: [db]	
restart: unless-stopped	
db:	
image: postgres:16	
volumes: [pgdata:/var/lib/postgresql/data]	
volumes:	
pgdata:	
网络 / 卷	
docker network ls	看网络
docker volume ls / prune	列 / 清未用
docker system prune -a	清未用镜像/容器/卷(慎)
Pro Tips	
镜像层缓存: 把 COPY package.json 放 npm install 前	
用 distroless / alpine 基础镜像减少攻击面	
不要在镜像里放 secret,用 env / secret manager	
多架构构建用 docker buildx build --platform linux/amd64,arm64	
本地调试镜像用 dive image 看每层占用	